

SESSION 2: BOOK KEEPING & CODE DISTRIBUTION

Version control using git, hosting code repositories at github and zenodo

Kosmas Hench

2024-03-08

Remember our need to convince our audience about the soundness the analysis behind “Figure 2”? Besides sharing the final working code (Session 1), it helps our cause if we can show that an actual human sat down and developed the code with the analysis of our specific data in mind: What we want is a kind of paper-trail that testifies to the work that we have put into the analysis – a kind of digital lab book. A very effective way to create such a paper-trail is to use version control during the development of your code. And nowadays, “version control” is basically synonymous with using the program git. In addition to keeping track of your progress and thought process (and yes, also of all the embarrassing typos, bugs and errors you make along the way), version control comes with several additional benefits beyond the mere track record: It allows you easily share your code, to travel in time and to parallel worlds! (well, sort of...)



created using dale3

1. USING THE PROGRAM git

This session focuses on a specific program for version control called **git**. **git** is a command line program that ships with most unix-systems (mac/linux) and is also available on windows. In this workshop we are going to refer to the bare-bones command line commands for pure **git**. Note however, that **git** is heavily used, and its popularity has led an integration of **git**-management in many software editors (IDEs), such as **RStudio** or **VScode**. This means that in practice many of the somewhat cumbersome steps described in the following can easily be ticked off on the side while working on the code for an analysis.

```
code_folder
├── script_final_for_real.R
├── script_final.R
├── script.R
└── data
    ├── data_final.tsv
    ├── data_fixed.tsv
    └── data.tsv
```

2. VERSION CONTROL AS DIGITAL LAB BOOK

Before we start diving into **git**, let's first talk about what we are trying to avoid: we *don't* want conflicting versions of the same file within our project – that sort of ambiguity leads to confusion. For all the files within our project, we want only one version, and that one is supposed to be the “real deal”. So, no previous attempts and drafts that might be mistaken as the data or scripts used to run the reported analysis. However, we just argued that we do want to keep track of the previous states of our project. To keep things clean, we turn our project folder into a **git repository** (or **repo**): a *folder with a memory* that keeps track of the development of files *internally* (the **git** command to do so is **git init**).

The basic idea of **git** is to take consecutive snapshots of the state of your project folder over time. In **git** lingo, taking a snapshot is called to create a **commit**. You can think of this a little like saving in a computer game (rather than like saving a file). That is because **git** remembers *all* your commits (and not just the most recent one). Yet a **commit** does not happen automatically: you will need to decide yourself *when* you want to take a snapshot of your project. Also, you can specify *which files* should be included in the next snapshot, so you can fine-tune what part of your progress is being logged. The terminology for including files in the upcoming **commit**, is to

Repository: In the case of **git**, a “repository” is basically a glorified name for a folder that contains a hidden folder called **.git** (which is where the book-keeping happens). If you delete this folder, you erase all the entries of the version control and your project ceases to be a **git** repository.

stage the file (the `git` command is `git add <filename>`). After having staged your selection of files, the snapshot is taken with the command `git commit -m "<commit label>"`. Note, that each commit is tagged with a commit message as label, which should describe why the snapshot was taken.



Each of these commits represents the archived state of your repository at a specific point in time. Every time you commit changes in your repository, `git` will record all changes in the staged files on a line-by-line basis. So `git` does not just remember which files changed, but also how they changed – however, this really only works well for plain text files (such as coding scripts).



In general, the `git` commands that we have covered so far (`init`, `add`, `commit`) are all that is really needed. For the basic version control workflow you want to first a) turn your project folder into a `git` repository and then b) edit the files therein, stage and commit the changes, and then rinse and repeat.

! What not to track ! While the whole version control section is all about communicating transparently, there are some aspects that we in fact do not want to share with the public. Typically, these are cases where transparency is in conflict with personal data privacy or any type of security concern (think e.g. personal communication with collaborators, or precise locations of an endangered and poached species). Other files that are part of our project that we typically do not want to include in the version control are large files, like raw data that is not in plain text format – particularly if these files are shared in some other way (SESSION 1). To exclude specific files (or folders) from the bookkeeping you can create a plain text file called `.gitignore`, which is simply a list of the file names that are then ignored by `git`.

3. VERSION CONTROL AS TIME MACHINE

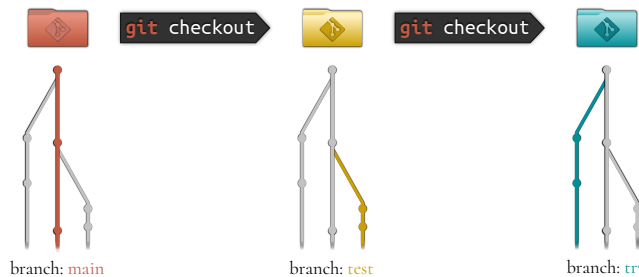
Once a `git` repository contains several commits, it is possible to restore the state from a committed stage of the project using the `git` command `git checkout <commit_id>`. This function allows you to do look up previous states of your analysis, which can be very useful in case you botched something up and want to revert to an earlier, working version. It also enables you to label specific versions of interest that can be inspected later (eg. the state of the analysis at the initial submission of your study, after the first round of revisions, after acceptance, etc).

4. SAFE DEVELOPMENT WITHIN BRANCHES

We now know we can revert to previous stages if we messed up our repository, which is reassuring. Yet, sometimes we might already know ahead of time that we need to embark on a more experimental journey with the design for our analysis, and we might be concerned that this exploration will break the currently working code of our analysis. For these cases we can create a save environment to draft the next steps in a secure copy of the repository: we can create a `git branch` using the command `git branch <branch_name>`. Within a side branch, it is possible to work on files in

Branch: An internal copy of the repository. The default branch that is created upon initializing a `git` repository is typically called either "main" or "master". Several branches can exist in parallel, and you can switch between them using `git checkout <branch_name>`.

without modifying the original versions in the "main" branch. To toggle between branches, you can again use the `checkout` command, this time specifying the branch name: `git checkout <branch_name>`. By switching between branches, the actual files within your repository *will change*. This means, for example if a file exists only in the "test" branch, but not in "main", it will look like this file was deleted when activating the "main" branch — re-activating the "test" branch will restore that file though.



At some point the exploratory part of the design can hopefully be successfully concluded, and you might want to import the changes made in the developing branch into the "main" branch. What you need to do here is to **merge** the two branches: Starting from the "main" branch (the branch into which we want to incorporate the updates), we can import the changes from another branch using `git merge <branch_name>`.

Sometimes when merging branches, issues can arise that lead to *merge conflict*. This happens when a file has changed in both branches and `git` is not able to tell which version (or what sort of compromise) is the desired one to keep for the future. These situations need to be resolved manually, which requires a combination of a `add` and `commit` command (we will work through such a scenario in the hands on section).

5. GIT-RELATED INFRASTRUCTURE FOR CODE SHARING

So far all the action has taken place only at our local machine (aka. your laptop). However, once we have turned a folder into a `git` repository, we can easily share it with other people by connecting it with *github*. As the name implies, github is a platform where you can deposit `git` repositories online, where other people can easily access your code and that comes with a lot of nice tools to inspect the repository's history straight out of the box. In `git` lingo what we want to do is to set up a **remote** for the repository on github (we will go through the required steps in the hands-on part). In a nutshell, we need to set up a location for the repository on our github account, which will provide us with a url that will specify the exact location of the remote. This connection is made from the local version of the repository (on your laptop) with the command `git remote add origin https://github.com/<url>` (here we name the remote "origin", which is the default name for the github connection).

Once we have established a remote, we can synchronize the local and the remote instances by using `git push origin <branch_name>` (laptop → github) and `git pull origin <branch_name>` (laptop ← github). Repositories that are hosted on github can also directly be downloaded to a local machine with `git clone <repository_url>`. In this case, the connection to the original github repository is going to be setup already (again using the name "origin" for this remote by default). The idea with github is to have an online representation of your code that is in sync with the local version that you are working on. The github repository is thus a dynamic place that will change over time. This is desired for software that needs to be developed and maintained over time (and for which `git` was originally intended), but for the scientific record we want to be able to archive particular snapshots of your code in a more

Remote: An online version of the repository. Typically, this is a copy on **github**, which is synchronized with the local version of the repository. This makes it easy to share your code with others, or to transfer and synchronize your code between several computers.

static way. Luckily, it is easy to connect [zenodo](#) to a github repository, which allows you to cleanly archive selected commits and provides you with a citable DOI for those (which you can then use to refer to the code repository within a publication). So to provide a lasting reference for your study's code base, it is good practice to deposit the code at zenodo, typically with a “living copy” at hosted at github.

Recap

To effectively use version control on a coding project, all that is needed are a couple of commands that record the progress of the code contained within a designated folder:

Any folder can be turned into a git repository with `git init`.



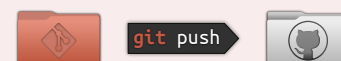
Progress within the repository is logged with `git commit`, which creates timestamps of the project that can be restored. Only staged files (`git add <file>`) are being archived.



The command `git checkout` can be used to switch between branches or to revert to a previous commit.



If the local repository is mirrored on github, local changes can be updated on github using `git push <remote> <branch>`. (In this case the github repository is a remote of the local repository).



When the remote contains changes that need to be update locally, this can be done via `git pull <remote> <branch>`.



To exclude sensitive information of large data files from the version control, these can explicitly listed within a blacklist called `.gitignore`.

FURTHER READING

- Braga, P.H.P. *et al.* (2023). Not just for programmers: How GitHub can accelerate collaborative and reproducible research in ecology and evolution. *Methods in Ecology and Evolution*, 14(6):1364–1380. DOI: [10.1111/2041-210X.14108](#).
- Bryan, J. (2018). Excuse me, do you have a moment to talk about version control? *The American Statistician*, 72(1):20–27. DOI: [10.1080/00031305.2017.1399928](#).